

TASK 15

1. Fair Scoring Improvements Module

fairness_engine.py

import re

def normalize_scores(candidate_scores):

scores = [

c["final_score"]

for c in candidate_scores

]

min_score = min(scores)

max_score = max(scores)

Avoid division by zero

if max_score == min_score:

return candidate_scores

for c in candidate_scores:

c["normalized_score"] = round(

(

(

c["final_score"] - min_score

)

/

(

max_score - min_score

)

) * 100,

2

)

```

    return candidate_scores

def mask_sensitive_data(candidate):
    sensitive_fields = [
        "name",
        "gender",
        "age",
        "photo",
        "location"
    ]
    for field in sensitive_fields:
        if field in candidate:
            candidate[field] = "MASKED"
    return candidate

def reduce_keyword_bias(
    skill_score,
    semantic_score
):
    adjusted_score = (
        0.6 * semantic_score
        +
        0.4 * skill_score
    )
    return round(adjusted_score, 2)

def generate_fair_score(candidate):
    skill_score = candidate.get(
        "skill_score",
        0

```

```

)
semantic_score = candidate.get(
    "semantic_score",
    0
)
fair_score = reduce_keyword_bias(
    skill_score,
    semantic_score
)
candidate["fair_score"] = fair_score
return candidate

```

2. Normalisation Logic

parsers/normalization.py

```

import re

def normalize_resume_text(text):
    text = text.lower()

    # Remove special characters
    text = re.sub(
        r"^[^a-z0-9\s\.\_\-]",
        "",
        text
    )

    # Normalize spaces
    text = re.sub(
        r"\s+",
        " ",
        text
    )

```

```
)
```

```
# Standardize headings
```

```
replacements = {  
    "professional experience":  
        "experience",  
    "work experience":  
        "experience",  
    "academic background":  
        "education",  
    "skill set":  
        "skills"  
}
```

```
for key, value in replacements.items():  
    text = text.replace(  
        key,  
        value  
    )
```

```
return text.strip()
```

```
tests/test_fairness_engine.py
```

```
from ats_engine.fairness_engine import (  
    normalize_scores,  
    mask_sensitive_data,  
    generate_fair_score  
)
```

```
def test_fairness_engine():
```

```
    candidates = [
```

```

{
    "candidate_id": "C1",
    "final_score": 90,
    "skill_score": 85,
    "semantic_score": 88,
    "name": "John"
},
{
    "candidate_id": "C2",
    "final_score": 70,
    "skill_score": 65,
    "semantic_score": 75,
    "name": "Alice"
}
]

candidates = normalize_scores(
    candidates
)

for c in candidates:
    mask_sensitive_data(c)
    generate_fair_score(c)

print(
    "✅ Fairness engine test passed!"
)

print(candidates)

test_fairness_engine()

```

main.py

```
from ats_engine.fairness_engine import (  
    normalize_scores,  
    mask_sensitive_data,  
    generate_fair_score  
)
```

```
from parsers.normalization import (  
    normalize_resume_text  
)
```

♦ Step 7: Resume Normalization

```
normalized_text = normalize_resume_text(  
    text  
)
```

```
print("\nNORMALIZED RESUME:\n")
```

```
print(normalized_text[:300])
```

♦ Step 8: Fairness Engine

```
fair_candidates = [  
  
    {
```

```

        "candidate_id": "C1",
        "final_score": 88,
        "skill_score": 85,
        "semantic_score": 82,
        "name": "John",
        "gender": "Male",
        "location": "NY"
    },

    {
        "candidate_id": "C2",
        "final_score": 72,
        "skill_score": 70,
        "semantic_score": 75,
        "name": "Alice",
        "gender": "Female",
        "location": "LA"
    }
]

```

```

fair_candidates = normalize_scores(
    fair_candidates
)

```

```

for c in fair_candidates:

```

```

    mask_sensitive_data(c)

```

```
generate_fair_score(c)
```

```
print("\nFAIRNESS RESULT:\n")
```

```
print(  
    json.dumps(  
        fair_candidates,  
        indent=2  
    )  
)
```

```
output
```

```
[  
  {  
    "candidate_id": "C1",  
    "final_score": 88,  
    "normalized_score": 100,  
    "fair_score": 83.2,  
    "name": "MASKED",  
    "gender": "MASKED",  
    "location": "MASKED"  
  }  
]
```

```
scoring/fairness_report.txt
```


3. BIAS Reduction Documentation

Fairness & Bias Reduction Engine – Zecpath AI

OBJECTIVE

The objective of this module is to improve fairness and consistency in ATS-based resume evaluation by:

- Normalizing resume content into a standard format
- Reducing over-dependence on keyword matching
- Applying score normalization across candidates
- Masking sensitive personal attributes
- Supporting fair and explainable candidate evaluation

MODULES IMPLEMENTED

1. Fair Scoring Engine

File:

ats_engine/fairness_engine.py

Features:

- Score normalization
- Sensitive data masking
- Keyword bias reduction
- Fair score generation

Key Functions:

normalize_scores()

Applies min-max normalization across candidate scores.

mask_sensitive_data()

Masks sensitive attributes such as:

- Name
- Gender
- Age

- Photo
- Location

`reduce_keyword_bias()`

Balances semantic similarity with keyword matching.

`generate_fair_score()`

Generates a fairness-aware score using:

- Skill score
- Semantic similarity score

2. Resume Normalization

File:

`parsers/normalization.py`

Features:

- Lowercase conversion
- Removal of special characters
- Space normalization
- Standardized section headings

Example Heading Conversion:

Professional Experience → Experience

Work Experience → Experience

Academic Background → Education

Skill Set → Skills

FAIRNESS TECHNIQUES USED

1. Resume Standardization

All resumes are converted into a common structure before evaluation.

Benefits:

- Reduces formatting bias
- Improves consistency
- Helps semantic matching

2. Bias Reduction

Sensitive personal attributes are masked before scoring.

Masked Fields:

- Name
- Gender
- Age
- Photo
- Location

Benefits:

- Reduces demographic bias
- Encourages skill-based evaluation
- Improves fairness

3. Semantic Weighting

Instead of relying only on keyword frequency, the system blends:

- Keyword skill score
- Semantic similarity score

Formula:

$$\begin{aligned} \text{Fair Score} = & \\ & (0.6 \times \text{Semantic Score}) \\ & + \\ & (0.4 \times \text{Skill Score}) \end{aligned}$$

Benefits:

- Handles synonyms

- Improves contextual understanding
- Reduces keyword stuffing advantage

STRUCTURED OUTPUT EXAMPLE

```
{  
  "candidate_id": "C123",  
  "final_score": 82,  
  "normalized_score": 78.5,  
  "fair_score": 80.2,  
  "masked_profile": {  
    "name": "MASKED",  
    "gender": "MASKED",  
    "location": "MASKED"  
  }  
}
```

ADVANTAGES

- Fair candidate evaluation
- Transparent scoring
- Reduced recruiter bias
- Standardized resume processing
- Better semantic understanding

LIMITATIONS

- Initial version uses fixed fairness rules
- Semantic models may still contain hidden bias
- Resume extraction quality affects fairness

FUTURE IMPROVEMENTS

- AI-based bias detection
- Adaptive fairness tuning
- Recruiter feedback integration
- Cross-language fairness support
- Domain-specific fairness optimization

DELIVERABLES COMPLETED

1. Fair scoring improvements
2. Resume normalization logic
3. Bias reduction logic
4. Fairness documentation